Informe de práctica de la semana 7

Modelado UML y control de estados

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S7-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Documentar el proyecto del hotel mediante clases, dos secuencias y estados, comprobar su correspondencia con el código y preparar el paquete técnico EVI1.

El ejemplo de máquina de estados modifica el motivo de suspensión antes de comprobar si la transición es válida. Se corrigió el orden: validar y transicionar primero, actualizar el motivo después. El UML debe representar la vida y cardinalidad reales; un vector por sí solo no garantiza una multiplicidad mínima de uno.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
1 Clases	docs/uml/	Modelo elaborado, generación desde Python y C++ y comparación documentada.
2 Secuencias	secuencia_CU01 y secuencia_CU02	Crear y anular reserva con participantes, mensajes, alt y opt.
3 Estados	python/estados.py y cpp/estados.hpp	Máquinas genéricas y transición integrada en ReservaHotel.
4 Revisión	docs/revision_tecnica.md	Autorrevisión asistida y ficha separada para el compañero.
5 EVI1	README y paquete de integración	Fuentes, pruebas, UML e informe de decisiones enlazados.

Caso explicado

Una matrícula recién creada está REGISTRADA. Intentar suspenderla directamente es inválido: se rechaza la transición y permanecen el estado inicial, el motivo vacío y una sola entrada de historial. Esta prueba comprueba que validar y registrar la transición ocurre antes de guardar el nuevo motivo.

Conceptos para interpretar el caso

Una máquina de estados enumera los cambios permitidos de una entidad. Una guarda es una condición adicional de la transición. La multiplicidad UML expresa cuántos objetos pueden participar en una relación; no se deduce únicamente del tipo de colección.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

El modelo de clases distingue entidades, repositorios y servicios. La reserva mantiene una máquina de estados propia; huésped y habitación tienen existencia independiente. ServicioReservas recibe repositorios y contratos de cotización y notificación.

Crear una reserva consulta huésped y habitación, comprueba cruces de fechas, cotiza, confirma e inserta. Anular consulta la reserva y solicita una transición; la disponibilidad se deriva de las reservas no anuladas y no se guarda como una bandera duplicada.

Los estados del hotel son REGISTRADA, CONFIRMADA, HOSPEDADA, FINALIZADA y ANULADA. FINALIZADA y ANULADA no admiten salidas. Las transiciones inválidas dejan intactos estado e historial. El ejemplo académico mantiene REGISTRADA, VIGENTE, SUSPENDIDA y ANULADA.

Notación y comprobaciones importantes

Elemento	Aplicación en el hotel	Interpretación UML o C++
Herencia	Habitacion a sus variantes	Triángulo vacío hacia la base
Realización	TarifaEstandar implementa contrato	Línea discontinua hacia la interfaz
Composición	ReservaHotel posee su máquina	Rombo lleno en el propietario
Asociación	Reserva usa huésped y habitación	Existencia independiente y cardinalidades explícitas
Multiplicidad	Colección puede estar vacía	0..* salvo invariante más restrictivo
Const y noexcept	Son propiedades distintas	Una consulta const todavía puede lanzar

Cómo seguir la implementación

Python describe las transiciones admitidas en una tabla y C++ emplea estados de enum class. En ambos casos se valida el par de estados y su guarda antes de modificar el estado o el historial. La reserva transforma los errores de transición en ErrorEstadoReserva, que pertenece al dominio del hotel.

3 Verificación y resultados

Las pruebas registradas aprobaron 9 casos Python y 8 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	9 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	8 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Casos representativos comprobados

Caso	Resultado observado	Qué demuestra
Suspender una matrícula REGISTRADA	Rechazo; motivo vacío e historial de longitud 1	Una transición inválida no deja cambios parciales.
Guarda de pago con condición falsa	El estado A se conserva	La condición se evalúa antes de pasar a B.
Reserva confirmada ingresada y finalizada	FINALIZADA; anulación posterior rechazada	El estado final impide nuevas operaciones del ciclo.
Anular una reserva del hotel	Tres habitaciones disponibles para el intervalo	La disponibilidad se deriva de las reservas activas.

La consistencia del modelo requiere revisar las llamadas y la propiedad de los objetos, además de generar imágenes. pyreverse y Doxygen aportan inventarios; las cardinalidades y la composición se contrastan con los contratos de las entidades.

Lectura de la salida

MATRICULA=ANULADA y HISTORIAL_MATRICULA=5 incluyen el estado inicial y cuatro transiciones válidas. RESERVA=FINALIZADA y su historial de cuatro entradas corresponden al estado inicial, confirmación, ingreso y finalización.

4 Aplicación al hotel y conclusiones

Los diagramas de crear y anular reserva siguen llamadas existentes en ServicioReservas, Repositorio, ReservaHotel, MaquinaEstados y el notificador simulado. La tabla de trazabilidad vincula cada mensaje con el método y archivo. El diagrama de estados coincide con los cinco estados del enum y sus cinco transiciones hoteleras.

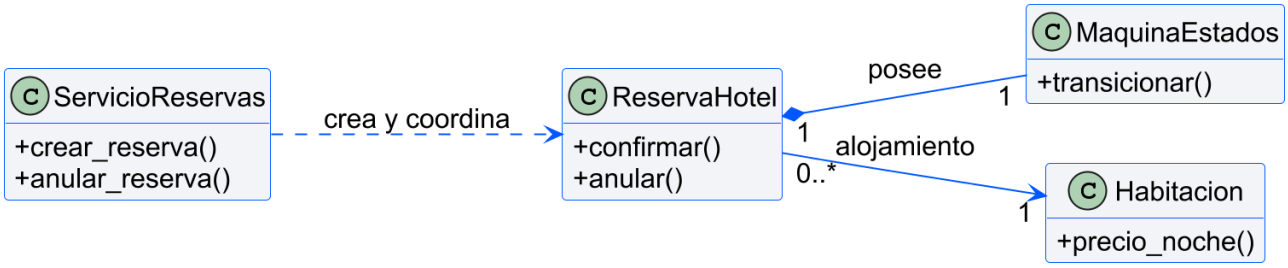


Figura 1 Detalle de composición y asociaciones del hotel. El modelo completo, las secuencias y los estados están en docs/uml.

Conclusión de la práctica

Los modelos y las pruebas describen el mismo ciclo de vida y los rechazos conservan el estado previo. La reserva posee su máquina de estados y referencia entidades que existen de forma independiente. La trazabilidad permite localizar en el código cada mensaje de las secuencias.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 7*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.